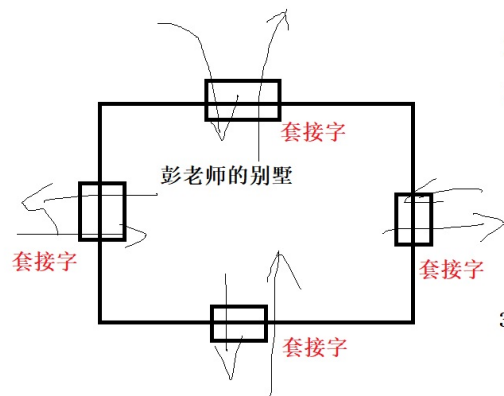


一.引入多路复用

多路复用能让一个进程同时监控多个文件描述符（比如网络套接字、文件、管道等），并阻塞等待其中任意一个 / 多个描述符就绪（可读 / 可写 / 异常），避免进程对每个描述符单独轮询或创建大量线程，从而高效管理 I/O 操作。



1. 四个门随时都有可能进出人

2. 为了方便招待这些访问的客人

方案1: 请四个管家—人工成本贵

方案2: 安装四个监控, AI

对应计算机创建四个线程

多路复用

3. 生活中例子:

门—用来人进出

计算机:

IO通道—用来数据的 **进出**

可以用来进行数据收发的, 都是IO通道

数据的发送 `send` `write` `sendto`
数据的接收 `recv` `read` `recvfrom`

4. 多路复用: 类似于监控, 用来检测套接字是否有数据进出

套接字有数据可读—进

套接字有数据可写—出

生活中的监控—监测门是否有人进出

计算机多路复用—监测套接字是否有数据进出

二.多路复用相关的接口函数

(1)核心接口函数select

```
#include <sys/select.h>
```

```
int select(int maxfd, fd_set *readset, fd_set *writeset, fd_set *exceptset, const struct timeval *timeout)
```

返回值: 成功 >0

超时 ==0

失败 -1

参数: maxfd -- 所有要监测的文件描述符中最大的文件描述符+1

readset -- 你想监测文件描述符是否有数据可读(专业术语叫做监测文件描述符是否读就绪)

writeset -- 你想监测文件描述符是否有数据可写(专业术语叫做监测文件描述符是否写就绪)

exceptset -- 你想监测文件描述符是否发生异常(专业术语叫做监测文件描述符是否异常就绪)

timeout -- 超时, 最小等待时间, 设置为NULL, 永久等待

```
struct timeval
{
    tv_sec; //秒
    tv_usec; //微秒
}
```

比如: `select(最大+1, &myset, NULL, NULL, 超时)` //我想监测myset中的文件描述符是否有数据流入(可读)

`select(最大+1, NULL, &myset, NULL, 超时)` //我想监测myset中的文件描述符是否有数据可写

(2)文件描述符集合操作的相关函数

```
void FD_ZERO(fd_set *fdset); //清空集合
```

```
void FD_SET(int fd, fd_set *fdset); //将fd加入集合fdset中
```

```
void FD_CLR(int fd, fd_set *fdset); //将fd从集合中删除
```

```
int FD_ISSET(int fd, fd_set *fdset); //判断fd在不在集合fdset中 在集合中返回1 不在集合中返回0
```

三.例子

以tcp单向通信代码为例(服务器发送信息给客户端)，现在我打算用多路复用实现

先分析服务器：

服务器--》发送信息给客户端 （用到键盘--》文件描述符是0）

再分析客户端：

接收来自服务器的信息 （用到套接字 --》套接字的文件描述符）